



ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR
Membre de
HONORIS UNITED UNIVERSITIES

L'écosystème Big Data

Chapitre 1 : Introduction au Big Data

Dispensé par: Dr. Asma Ouarhim
Enseignant/chercheur EMSI

1

La Révolution de la Donnée à l'Ère Numérique

L'écosystème Big Data ne se résume pas à une simple accumulation de serveurs ou à l'utilisation de nouveaux logiciels. C'est un changement de paradigme informatique et économique. Traditionnellement, nous adaptons nos données aux limites de nos machines ; aujourd'hui, nous concevons des architectures qui s'adaptent à la croissance infinie des données. Cette inversion du rapport de force entre l'infrastructure et l'information est le fondement de la transformation numérique des entreprises.

Point de réflexion : En 2025, on estime que 463 exaoctets de données seront créés chaque jour dans le monde. Pour donner une échelle, cela représente l'équivalent de 200 milliards de DVD par jour.

2

I. La Genèse : Pourquoi le Big Data ?

L'émergence du Big Data est le résultat d'une convergence entre trois phénomènes :

L'explosion des sources : Capteurs IoT, réseaux sociaux, journaux de serveurs (logs), transactions financières et imagerie satellite.

La baisse des coûts de stockage : Le passage du stockage matériel propriétaire à des solutions de stockage distribué sur du matériel "commodité" (serveurs standards).

La limite des systèmes relationnels (SGBDR) : Les bases de données traditionnelles (SQL) ne peuvent plus supporter l'élasticité nécessaire ni la variété des formats non structurés.

3

1. L'explosion des sources de données

La diversité des producteurs de données a radicalement changé :

- **L'Internet des Objets (IoT)** : Capteurs industriels (Industrie 4.0), compteurs intelligents, véhicules connectés, montres connectées.
- **L'activité humaine numérique** : Réseaux sociaux (3 milliards de likes par jour sur Facebook), historiques de navigation, vidéos YouTube (500 heures de vidéo uploadées chaque minute).
- **L'activité systémique** : Journaux de serveurs (logs), transactions bancaires (plus de 500 000 par seconde chez Visa), imagerie satellite et médicale.

4

2. La baisse drastique des coûts de stockage

Le coût du stockage est passé d'environ 10 000\$ par gigaoctet dans les années 80 à moins de 0,02\$ aujourd'hui. Cette démocratisation a permis le passage :

- **Du stockage propriétaire** (matériel spécialisé et coûteux)
- **Au stockage distribué sur matériel "commodité"** (serveurs standards type x86)
- **Avec une nouvelle philosophie** : "Conservons toutes les données, on ne sait jamais quand elles pourraient servir"

5

3. La limite des systèmes relationnels (SGBDR)

Les bases de données traditionnelles (SQL) ont montré leurs limites face à trois défis :

- **L'élasticité** : Impossible de scaler horizontalement (ajouter des machines) facilement
- **La variété** : Incapacité à gérer efficacement les données non structurées (images, vidéos, textes longs)
- **La performance** : Ralentissement exponentiel face aux volumes massifs

Cas concret : En 2004, Google réalisait que pour indexer le web, il lui fallait soit un superordinateur à 50 millions de dollars, soit 10 000 serveurs standards à 5000 dollars chacun. Le choix du "commodity hardware" a donné naissance à leur révolutionnaire système de fichiers distribué.

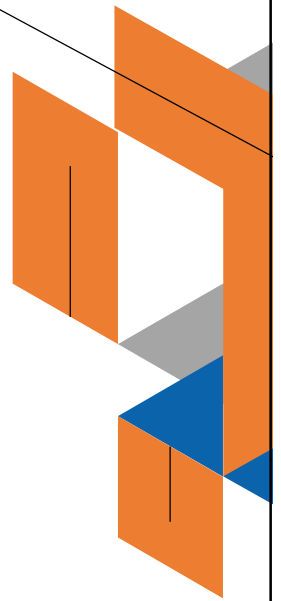
6

Traces numériques

- Les traces générées lors de l'utilisation de la technologie numérique sont composées de tout type de données saisies ou générées automatiquement.
- L'idée de base du Big Data est que les traces, de plus en plus volumineuses, de notre utilisation des outils numériques peuvent être **collectées, stockées** et **analysées** afin d'en tirer des connaissances pouvant aider à la **prise de décision**, la **prévision des risques** ou **l'amélioration des processus**, ...
- La trace numérique personnelle est un enregistrement de toutes les actions d'un individu sous forme de données informatisées, qui peut être consulté, trié, classé et diffusé.
- Appliquer à partir de plusieurs appareils et surtout sous un format groupé (avec d'autres individus sur des sites, plateforme, etc.), les traces deviennent publiques et utilisées par les autres.

Toute information numérique est une trace :

- ✓ Appel téléphonique, SMS, Chat, Mails, Applications de communications
- ✓ Navigation (recherche, visites, ...)
- ✓ Photos "partagées"
- ✓ Lecture/écriture de données (contenus, vidéos, musiques, ...)
- ✓ ...



7

Traces numériques

Chiffres:

Actuellement, chaque minute sur internet :

- +4 millions de recherches sont faites sur le moteur de recherche **Google**
- +1 million de personnes se connectent sur **Facebook**
- +2 millions de snaps sont envoyés
- +4,5 millions de vidéos sont vues sur **Youtube**
- +100 millions de mails sont envoyés
- ...

Exemples de volumétrie (1 minute)

- ✓ 3,3 millions de publications sur **Facebook** sont créées
- ✓ 66 000 photos **Instagram** sont téléversées
- ✓ 448.800 tweets sont créés
- ✓ 500 heures de vidéos **YouTube** sont téléversées
- ✓ On estime qu'il faut 1 année pour regarder sans arrêt des vidéos **Youtube** mises en ligne pendant 1 journée

- Chaque jour, environ 400 exaoctets (soit 0,4 zettaoctets) de données sont générées.
- 90% des données ont été créées au cours des deux dernières années



8

Définition

- Le cabinet d'études Gartner définit le Big Data comme suit:

"le Big Data est une forte volumétrie, haute Vitesse et grande Variété de données qui exigent des techniques innovantes et rentables de traitement d'information pour une meilleure compréhension, une prise de décision et l'automatisation et amélioration des processus."

- Ernst and Young propose la définition suivante:

*"Big Data refers to the **dynamic, large and disparate volumes** of data being created by people, tools and machines. It requires new, innovative, and scalable technology to collect, host and analytically process the vast amount of data gathered in order to derive **real-time business insights** that relate to consumers, risk, profit, performance, productivity management and enhanced shareholder value."*

9

Définition

Lisa Arthur (Forbes contributor) définit le **Big Data**:

"a collection of data from traditional and digital sources inside and outside a company that represents a source of ongoing discovery and analysis."

Quelques exemples d'utilisation du Big Data

- | | |
|---|---|
| <ul style="list-style-type: none"> • Science • Astronomy • Atmospheric science • Genomics • Biogeochemical • Biological • Social • Social networks • Social data <ul style="list-style-type: none"> * Twitter * Facebook * LinkedIn | <ul style="list-style-type: none"> • Commercial • Web / event / database logs • Sensor networks • Internet text and documents • Medical records • Photographic archives • Video / audio archives • Government • Military and homeland security surveillance |
|---|---|

10

Secteurs d'utilisation des Big Data

- ✓ Analyse du sentiment et de l'expérience client multicanal, ...
- ✓ Détection à temps des causes potentiellement mortelles dans les hôpitaux pour intervenir, ...
- ✓ Prédire les conditions météorologiques pour planifier l'utilisation optimale des éoliennes et optimiser les dépenses, ...
- ✓ Prendre des décisions de risque basées sur des données transactionnelles en temps réel, ...
- ✓ Identifier les criminels et les menaces provenant de sources vidéo, audio et de données disparates

Apport du Big Data

L'idée de base de la Big Data est que les traces, de plus en plus volumineuses, de notre utilisation des outils numériques peuvent être collectées, stockées et analysées afin d'en tirer des connaissances pouvant aider à la prise de décision, la prévision des risques ou l'amélioration des processus, ...

11

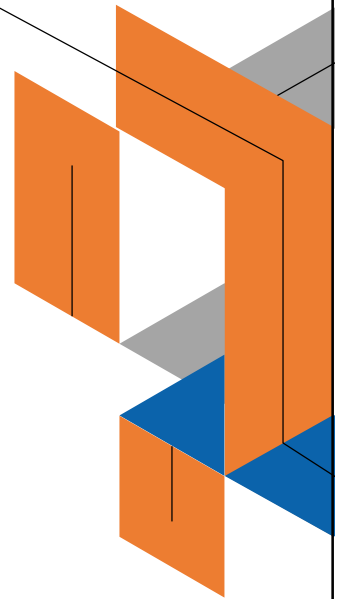
Exemples de volumétrie

- **2.5 petabytes:** Estimation maximale de la capacité mémoire du cerveau humain (1TB – 2.5PB)
- **13 petabytes:** quantité pouvant être téléchargée d'Internet en deux minutes, si environ 300 millions de personnes se trouvaient connectées simultanément.
- **4,75 exabytes:** séquences totales du génome de tous les habitants de la Terre.
- **124 exabytes:** quantité des données des data centers dans le monde en 2018
- **40 zettabyte:** quantité de données en 2020
- **200 zettabyte:** quantité de données en 2026

12

II. Les Dimensions Fondamentales : Au-delà des "V"

- Les **7V** ou les **5V** du Big Data ne sont pas l'œuvre d'un seul inventeur, mais une évolution progressive du modèle initial des **3V** défini par Doug Laney du cabinet **Gartner en 2001** (Volume, Vélocité, Variété).
- Ce modèle a été enrichi au fil du temps par des analystes et experts pour inclure Vérité, Valeur, Variabilité et Visualisation afin de mieux décrire la complexité des données.

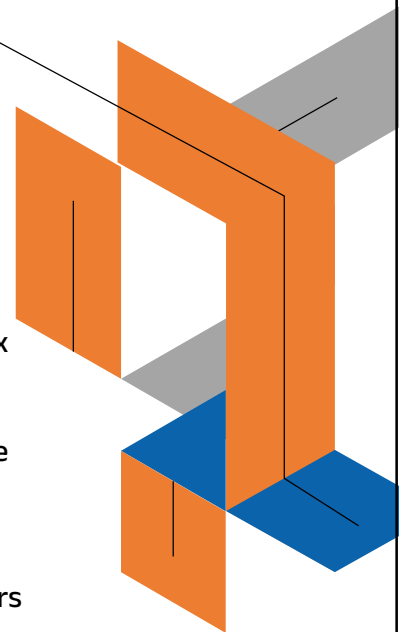


13

II. Les Dimensions Fondamentales : Au-delà des "V"

Si le Big Data est souvent défini par les **5V**, il est crucial d'en comprendre la profondeur opérationnelle et les implications concrètes :

- **Volume** : On passe de l'analyse d'échantillons à l'analyse de l'exhaustivité.
- **Vitesse** : La capacité à traiter les données en "streaming" (flux continu) pour prendre des décisions en temps réel.
- **Variété** : La coexistence de données structurées (tableaux), semi-structurées (JSON, XML) et non-structurées (vidéo, texte libre).
- **Véracité** : Le défi de la qualité de la donnée dans un océan de bruits et d'incertitudes.
- **Valeur** : L'alignement de la technologie sur les objectifs métiers (ROI).



14

II. Les Dimensions Fondamentales : Au-delà des "V"

Les 7V du Big Data définissent les caractéristiques clés des données massives pour les gérer et les valoriser efficacement : Volume, Vitesse, Variété, Véracité, Valeur, Variabilité et Visualisation. Ils décrivent la gestion de quantités astronomiques de données, leur traitement rapide, leur diversité, leur fiabilité et leur utilité stratégique.

Volume : Quantité massive de données générées (exaoctets), dépassant les capacités des systèmes de stockage traditionnels.

Vitesse (Vélocité) : Rapidité de création, de collecte et de traitement des données en temps réel ou quasi réel.

Variété : Diversité des formats de données : structurées, non structurées (vidéos, images), et semi-structurées.

Véracité : Fiabilité, précision et qualité des données, essentiels pour leur exploitation.

Valeur : Capacité à transformer les données brutes en informations exploitables pour la prise de décision.

Variabilité : Instabilité de la structure ou de la signification des données dans le temps, nécessitant des modèles adaptatifs.

Visualisation : Représentation graphique des données pour rendre les analyses complexes intelligibles et exploitables.

• (Parfois remplacés ou complétés par **Validité, Volatilité, ou Volume/Valeur**).

15

Différence clé : Les **5V** sont la base technique, alors que les **7V** englobent davantage les défis analytiques, la complexité contextuelle et la représentation des données pour la prise de décision.

1- Volume : De l'échantillon à l'exhaustivité

- **Approche traditionnelle** : On échantillonne pour réduire (on perd en précision)
- **Approche Big Data** : $n = \text{tout}$ (on analyse la population entière)
- *Exemple* : Google Translate n'apprend pas les règles grammaticales, il analyse des milliards de pages traduites pour trouver des patterns statistiques

16

2- Vitesse : Du batch au temps réel

- **Batch** : Traitement par lots (ex: facturation mensuelle)
- **Temps réel** : Streaming (ex: détection de fraude bancaire en moins de 100ms)
- **Nouvel enjeu** : La "péremption" de la donnée - une information non traitée dans la seconde peut perdre toute sa valeur

17

3- Variété : Le défi de l'hétérogénéité

- **Structurée** : Tableaux Excel, bases SQL (2% des données mondiales)
- **Semi-structurée** : JSON, XML, logs (18% des données)
- **Non-structurée** : Images, vidéos, audio, texte libre (80% des données)

18

4- Véracité : La confiance dans la donnée

- **Le paradoxe** : Plus on a de données, plus on a de "bruit"
- **Défis concrets** :
 - Données manquantes (trous de capteurs)
 - Données corrompues
 - Biais d'échantillonnage
 - Désinformation

19

5- Valeur : Le ROI de la donnée

- **La question fondamentale** : Comment transformer cette masse en avantage concurrentiel ?
- **Trois niveaux de valeur** :
 - **Descriptif** : Que s'est-il passé ? (Reporting)
 - **Prédictif** : Que va-t-il se passer ? (Machine Learning)
 - **Prescriptif** : Que doit-on faire ? (IA décisionnelle)

Données VS Information !

20

III. La Rupture Technologique : Le Calcul Distribué

- ✓ Le cœur du Big Data repose sur un principe fondamental : **"Diviser pour régner"** (*Divide and Conquer*).
- ✓ **Le paradigme révolutionnaire** : Plutôt que d'avoir une machine surpuissante et coûteuse (approche "scale-up"), on utilise un **cluster** (grappe) de machines standards (approche "scale-out").

Concept clé :

Au lieu d'apporter les données au code (ce qui sature le réseau), le Big Data apporte le code aux données. Le traitement s'effectue localement sur chaque nœud du cluster, minimisant ainsi les transferts massifs.

21

Le concept clé : "Data Locality"

L'innovation majeure du Big Data réside dans l'inversion du flux traditionnel :

- ✓ **Architecture classique** : On apporte les données au code (data → compute)
 - *Problème* : Saturation du réseau, latence, goulot d'étranglement
- ✓ **Architecture Big Data** : On apporte le code aux données (compute → data)
 - *Solution* : Le traitement s'effectue localement sur chaque nœud du cluster
 - *Avantage* : Minimisation des transferts réseau, parallélisation massive

❖ Illustration concrète

Pour traiter 1 pétaoctet de données (10^{15} octets) :

Approche classique : 10 jours (transfert + traitement)

Approche distribuée : 2 heures (traitement parallèle sur 1000 machines)

22

IV. Cartographie de l'Écosystème

L'écosystème Big Data est structuré comme une superposition de couches (la "Stack") où chaque couche a une responsabilité spécifique et des technologies adaptées :

Architecture en Couches

Couche	Rôle Principal	Technologies Phares	Cas d'Usage
Sources	Production des données	Applications, capteurs, réseaux sociaux, IoT	Collecte mondiale
Ingestion	Collecte et acheminement	Apache Kafka, Flume, Sqoop, NiFi	Pipeline temps réel
Stockage	Persistance massive	HDFS, Amazon S3, Azure Data Lake, HBase	Data Lake
Traitement	Transformation et calcul	Apache Spark (In-Memory), Flink, Hadoop MapReduce	ETL, Analytics
NoSQL	Schémas flexibles	MongoDB, Cassandra, Neo4j, Redis, HBase	Applications temps réel
Analyse/IA	Extraction de sens	Scikit-learn, TensorFlow, PyTorch, Power BI, Tableau	Data Science, Reporting
Gouvernance	Sécurité et qualité	Apache Atlas, Collibra, DataDog	Conformité RGPD

23

Zoom sur la Couche NoSQL

Le NoSQL répond à des besoins spécifiques que le SQL traditionnel ne couvre pas :

- **Document stores** (MongoDB) : Pour données JSON flexibles (profils utilisateurs)
- **Column stores** (Cassandra) : Pour écritures massives distribuées (logs, IoT)
- **Graph stores** (Neo4j) : Pour relations complexes (réseaux sociaux, recommandations)
- **Key-Value** (Redis) : Pour cache haute performance

24

V. Les Enjeux de la Gouvernance et de l'Éthique

Un écosystème Big Data n'est performant que s'il est maîtrisé. Les organisations font face à de nouveaux défis qui dépassent la simple technique :

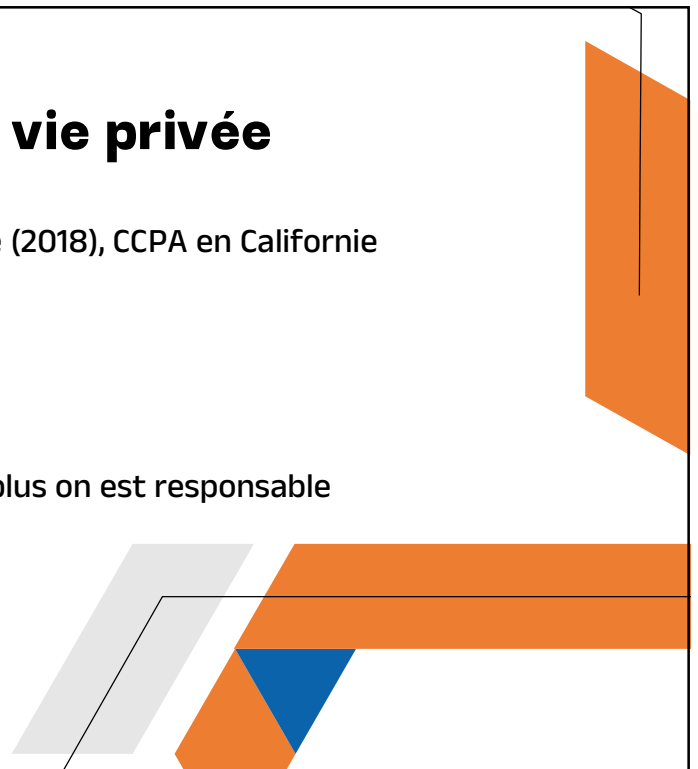
- **La protection de la vie privée** : Respect du RGPD et anonymisation des données.
- **Le Data Shadow (ou Dark Data)** : Éviter de stocker des masses de données inutilisées qui augmentent l'empreinte carbone et les risques de sécurité.
- **La démocratisation** : Passer de la "Data Science" réservée aux experts au "Self-Service Analytics" pour tous les collaborateurs.



25

1. La protection de la vie privée

- ✓ **Le cadre légal** : RGPD en Europe (2018), CCPA en Californie
- ✓ **Les obligations** :
 - Consentement explicite
 - Droit à l'oubli
 - Portabilité des données
 - Privacy by Design
- ✓ **Le paradoxe** : Plus on collecte, plus on est responsable



26

2. Le Data Shadow (ou Dark Data)

- ✓ **Définition** : Données collectées mais jamais analysées (60% à 80% des données stockées)
- ✓ **Les risques** :
 - **Écologique** : Empreinte carbone des data centers (3% de l'électricité mondiale)
 - **Financier** : Coût de stockage inutile
 - **Juridique** : Données conservées sans justification
- ✓ **Solution** : Mettre en place des politiques de cycle de vie des données

27

3. La démocratisation : La Data pour tous

- ✓ **L'évolution des rôles** :
 - **Data Scientist** : Expert technique, modélisation avancée
 - **Data Analyst** : Interprétation, visualisation
 - **Citizen Data Scientist** : Utilisateur métier avec outils no-code/low-code
- ✓ **Le concept de Self-Service Analytics** :
 - Donner aux métiers l'accès aux données sans intermédiaire technique
 - Outils comme Power BI, Tableau qui simplifient l'analyse
 - Formation massive des collaborateurs

28

4. Les biais algorithmiques

- ✓ **Le danger invisible** : Les algorithmes apprennent des biais humains présents dans les données historiques
- ✓ **Exemples concrets** :
 - Algorithmes de recrutement discriminants
 - Systèmes de crédit biaisés
 - Reconnaissance faciale moins performante selon l'origine ethnique
- ✓ **La solution** : Audit régulier des modèles, diversité dans les équipes de développement

29

Synthèse du Chapitre

Le Big Data n'est pas une mode technologique mais une transformation profonde de notre rapport à l'information. Ce premier chapitre a posé les fondations essentielles :

Comprendre le "pourquoi" : La convergence de facteurs technologiques et économiques

Maîtriser les concepts : Les 5V comme grille de lecture

Connaître l'architecture : Les couches technologiques et leurs rôles

Anticiper les défis : Éthique, gouvernance et responsabilité

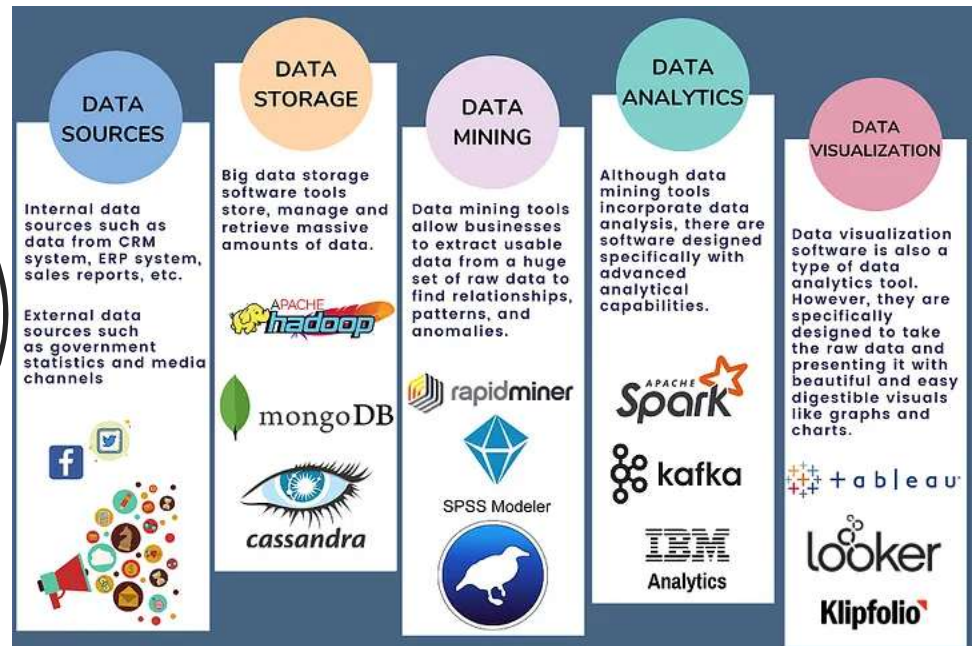


Ch2

Le prochain chapitre plongera dans le cœur du système avec Apache Hadoop, en détaillant son architecture, son système de fichiers distribué (HDFS) et son modèle de programmation (MapReduce).

30

A picture is
worth a
thousand
words



31



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de
HONORIS UNITED UNIVERSITIES

L'écosystème Big Data

Chapitre 2 :

L'Ecosystème Hadoop



Dispensé par: Dr. Asma Ouarhim
Enseignant/chercheur EMSI

32

Défis

- ✓ Parmi les grands problèmes liés au Big Data est la **lecture** (récupération) et **l'écriture** (stockage) des données.
- ✓ La vitesse de lecture et d'écriture des données doit être optimale pour qu'une solution Big Data soit adaptée à la vitesse d'évolution de la technologie et la volumétrie des données récoltées.
- ✓ Plusieurs serveurs pour stocker les données et plusieurs processeurs pour récupérer les données au même temps ne sont pas suffisants pour résoudre le problème. Il faut aussi des méthodes élaborées pour faire ces actions.
- ✓ → Une nouvelle manière de stocker
- ✓ → Une nouvelle façon de récupérer

33

Context

- ❑ La vitesse de lecture/écriture des disque s'est amélioré au cours des dernières années (*SSD VNMme peut atteindre 1500MB/s*). Cependant le temps de lecture/écriture des données de grande taille reste important.
- ❑ **Problème:** Combien de temps faut-il pour lire 2 To de données?

2 To (à 1000 Mb / sec):	{	1 disque - 2h 46min 10 disques - 16min 40 s 100 disques - 1 min 40 S 1000 disques - 10 sec
-------------------------	---	---
- ❑ **Solution:** Le traitement parallèle des données
 - **Calcul distribué:** plusieurs ordinateurs apparaissent comme un super ordinateur, communiquent les uns avec les autres par le passage de messages, opèrent ensemble pour atteindre un but commun.
- ❑ **Défis:** Hétérogénéité (matériel-OS) - Ouverture - Évolutivité - Sécurité - Concurrence - Tolérance aux pannes - Transparence

34

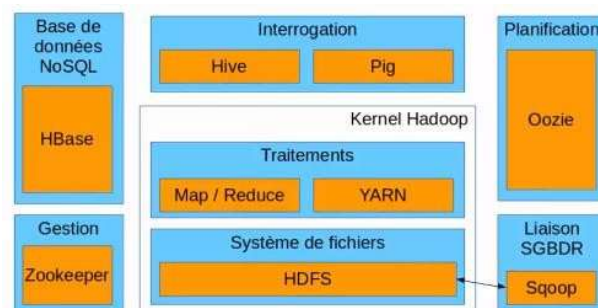
Historique:

- **2003** Google publie un document sur son "Google File Système" (GFS) un système de fichiers distribués pour gérer des larges volumes de données distribuées.
- **2004**: Mike Cafarella et Doug Cutting (*créateur de Lucene*) ont commencé à écrire un moteur de recherche distribué **Nutch** open source et basé sur Lucene (*son système de fichiers distribué NDFS*) pour pouvoir indexer plus de 1 Billion pages web.
- **2004**: Google a introduit *MapReduce* pour le traitement distribué.
- **2005**: Implémentation de *MapReduce* dans **Nutch**, tous les principaux algorithmes de **Nutch** fonctionnaient avec MapReduce sur des fichiers gérés par NDFS.
- **2006**: Projet **Hadoop** successeur de **Nutch** indépendant de Lucene.
- **2006**: Doug Cutting a rejoint Yahoo! qui a beaucoup investi pour transformer **Hadoop** en un système fonctionnant à l'échelle du Web.
- **Mai 2006**: Yahoo! déploie un cluster **Hadoop** de 300 nœuds pour son index de recherche.
- **2008**: Yahoo! a annoncé que son index de recherche était généré par un cluster **Hadoop** à 900 nœuds
- **Mai 2009**: Yahoo! a utilisé son cluster Hadoop pour traiter 1 To en 62 secondes par 1460 nœuds.
- **Juin 2009**: Yahoo rend le code source de Hadoop public.
- **Septembre 2009**: Doug Cutting rejoint **Cloudera**.

35

L'écosystème Hadoop

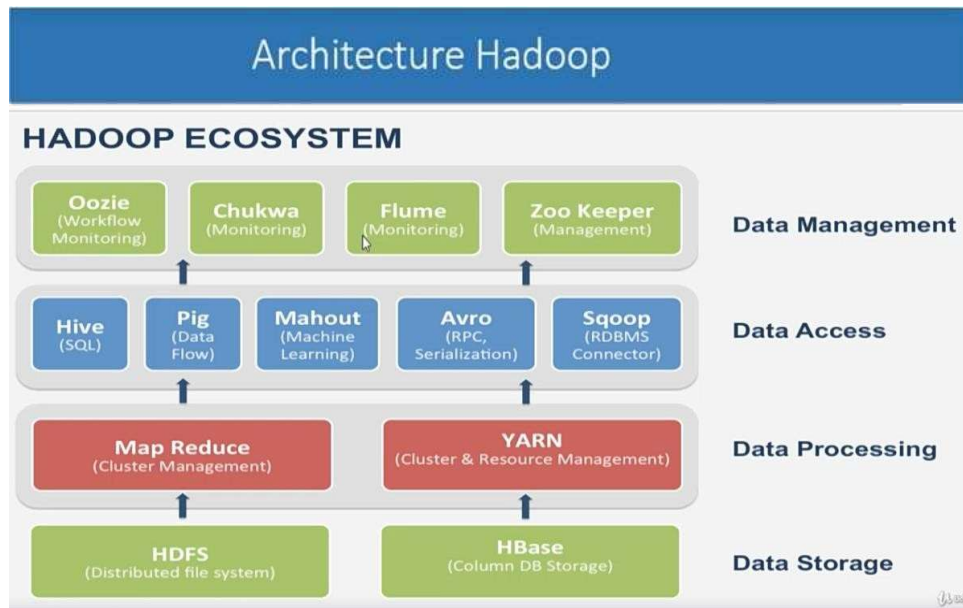
- ❑ Hadoop est basé sur les deux composants:
 - **MapReduce/YARN**: framework de **traitement** distribué
 - **HDFS** : pour un **stockage** distribué
- ❑ Hadoop est écrit en **Java** et sous License Apache
- ❑ Plusieurs projets sont liés à Hadoop: **Ecosystème Hadoop**



Simon Gilliot : Introduction à la plate-forme Hadoop et à son écosystème

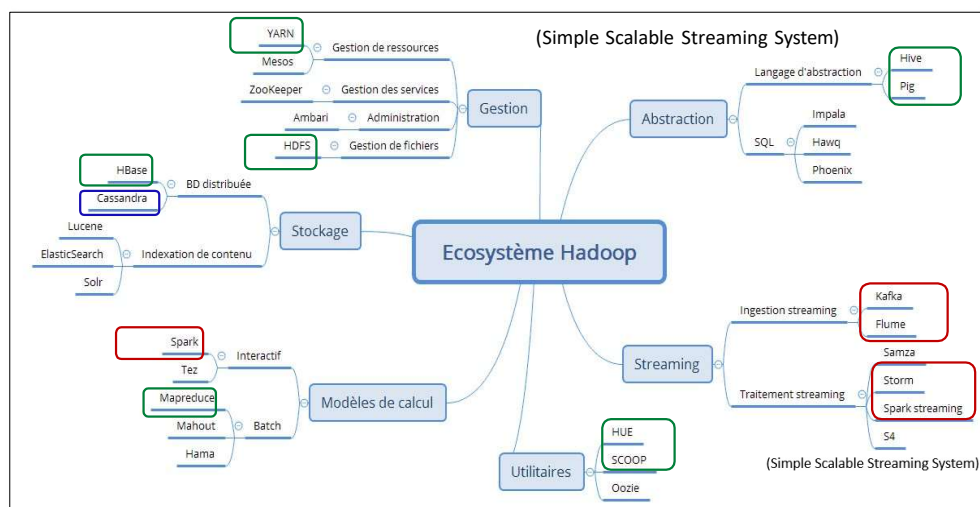
36

Architecture Hadoop



37

Architecture Hadoop



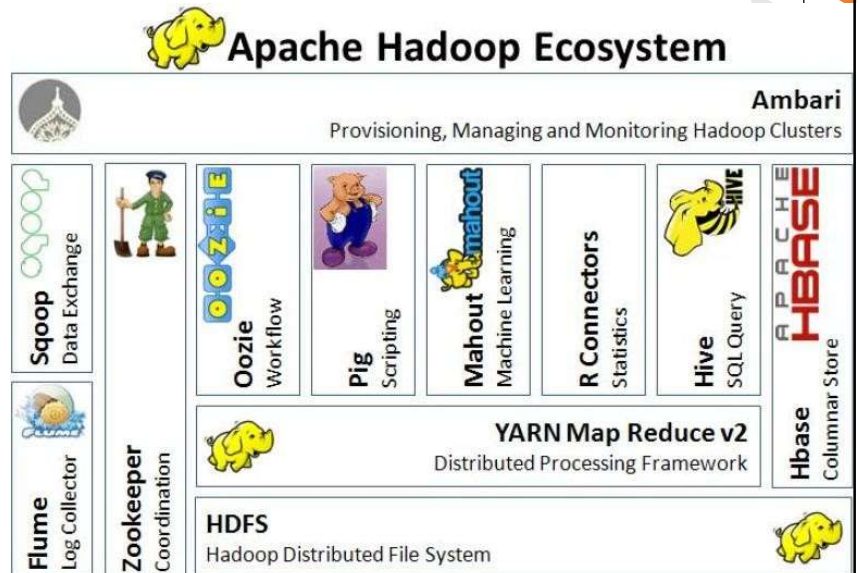
38

L'écosystème Hadoop

❑ Composants de base: **HDFS – MapReduce /Yarn**

❑ D'autres outils existant pour:

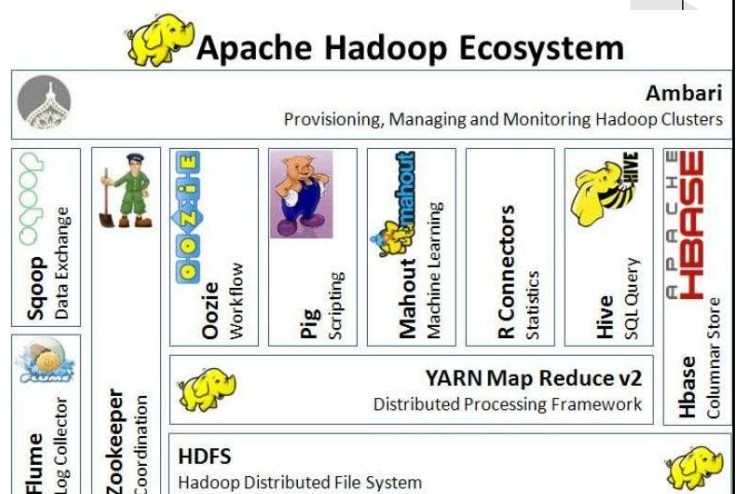
- La simplification des opérations de traitement sur les données
- La gestion et coordination de la plateforme
- Le monitoring du cluster



39

Composants de base d'un cluster Hadoop

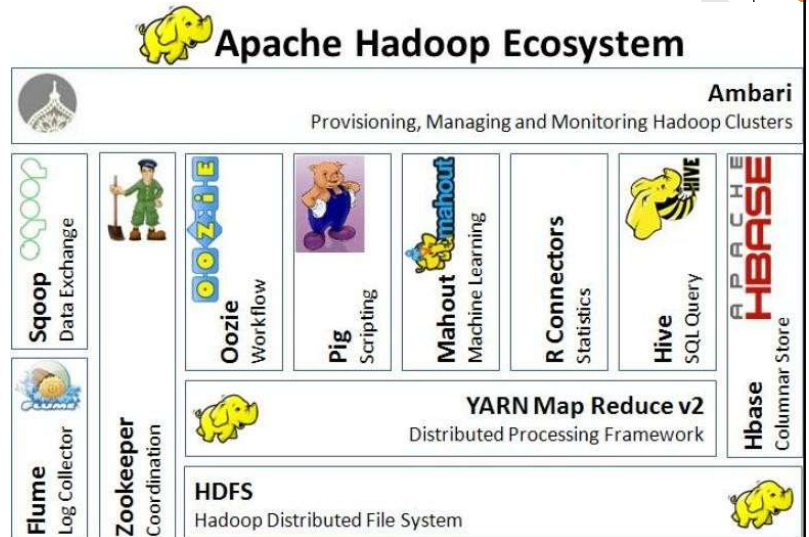
- Le système de fichier distribué, par exemple le HDFS d'Hadoop, qui gère le stockage distribué des données et fournit la tolérance aux pannes nécessaire lors de l'exploitation d'un cluster.
- Le modèle de calcul, comme MapReduce, qui est la façon dont les données sont parallélisés dans les noeuds du cluster.
- Le gestionnaire de ressources, comme YARN, qui permet de faire tourner plusieurs moteurs de calcul dans le cluster et d'exploiter son potentiel à son maximum.



40

L'écosystème Hadoop

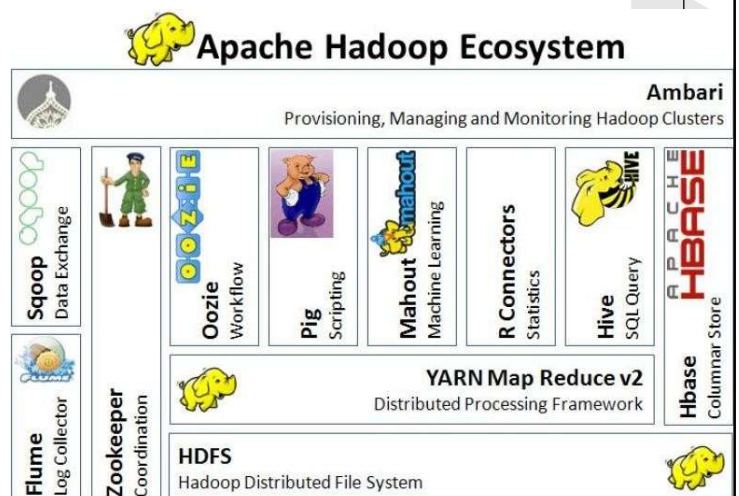
- ✓ **Pig:** Plateforme haut niveau pour le traitement de données, basée sur un langage de script Pig Latin
- ✓ **Hive:** Environnement de haut niveau pour le traitement de données, basé sur un langage proche de SQL
 - ✓ (Hive QL)
- ✓ **R Connectors:** permet l'accès à HDFS et l'exécution de requêtes Map/Reduce à partir du langage R
- ✓ **Mahout:** bibliothèque de Machine Learning et mathématiques
- ✓ **Oozie:** permet d'ordonner les jobs Map Reduce (Java, Python, Pig, Hive, ...), en définissant des workflows



41

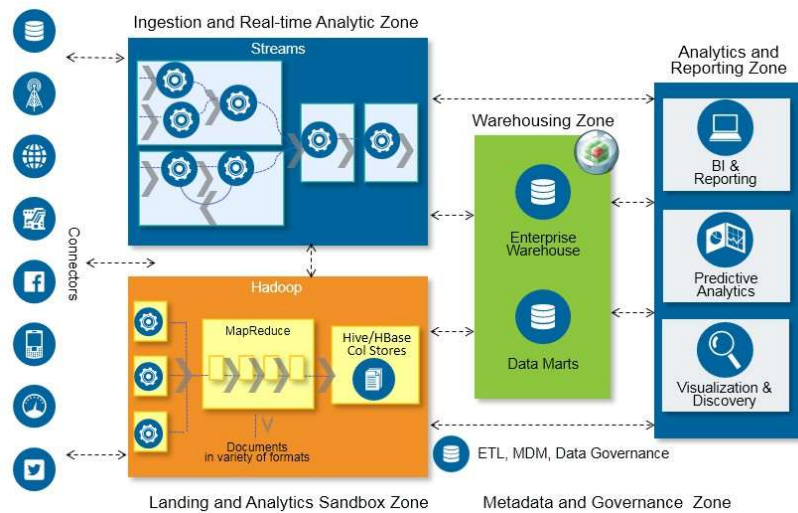
L'écosystème Hadoop

- ☐ **Hbase :** Base de données NoSQL orientée colonnes.
- ☐ **Sqoop:** Lecture et écriture des données à partir de BD externes
- ☐ **Flume:** Collecte de logs et stockage dans HDFS
- ☐ **Ambari:** outil pour la gestion et le monitoring des clusters
- ☐ **Zookeeper:** fournit un service centralisé pour maintenir les informations de configuration, de nommage et de synchronisation distribuée.



42

Hadoop dans les entreprises



Course Guide Volume 1: IBM BigInsights Foundation v4.0 (Course code DW613 ERC 1.0)

43

L'écosystème Big Data

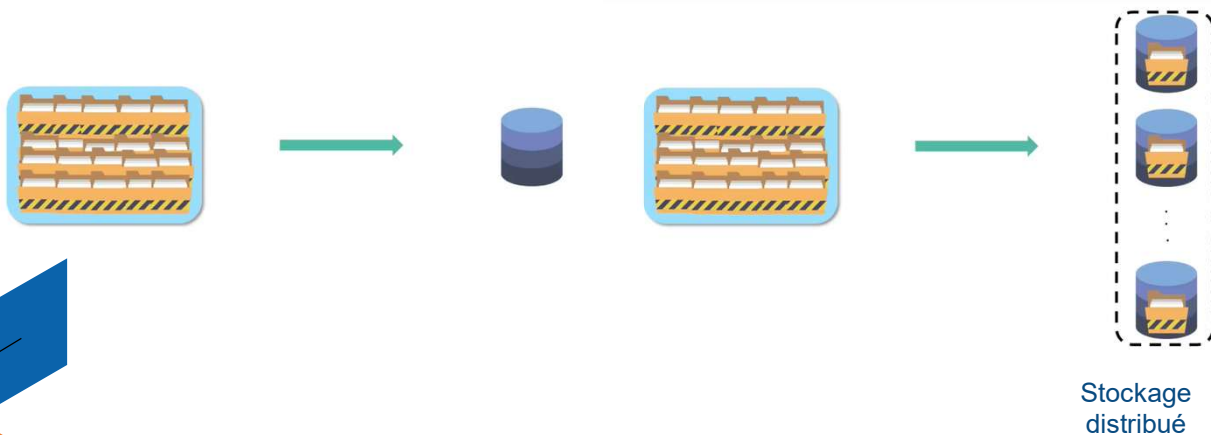
Chapitre 2 : I/HDFS



Dispensé par: Dr. Asma Ouarhim
 Enseignant/chercheur EMSI

44

Pourquoi HDFS



45

Hadoop: HDFS

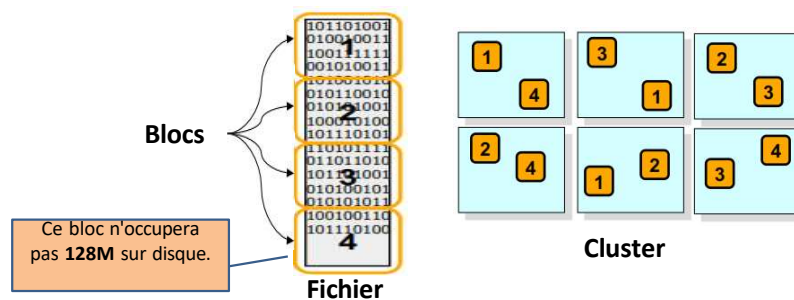
HDFS : Système de fichiers conçu pour stocker des fichiers très volumineux dans un cluster d'ordinateurs pas chers.

- ✓ **HDFS** stocke les données sur plusieurs nœuds qui forment un **cluster**
- ✓ Ecrire une fois/lire plusieurs fois (**WORM**: write once read many times): Les données sont généralement générées ou copiées à partir de la source. Des analyses (batch) sont ensuite effectuées sur cet ensemble de données au fil du temps. Chaque analyse impliquera une grande proportion, sinon la totalité, de l'ensemble de données.
- ✓ **HDFS réplique** les données sur plusieurs nœuds afin d'éviter leur perte en cas de panne de certains nœuds: **Fiabilité, Disponibilité**
- ✓ Chaque **fichier** est décomposé en plusieurs **blocs** qui seront **répliqués** sur les nœuds.

46

Hadoop: HDFS

- **Bloc**: taille par défaut 128 Mo depuis Hadoop 2.x (64 Mo sur Hadoop 1.x) (bloc OS = 4K octets par exemple)
- La taille est grande afin de minimiser le temps de recherche (localisation) des blocs.
- Possibilité de configurer cette taille lors de la création d'un fichier : Cette fonctionnalité est appelée "Variable Block Size" (VBS)
- Chaque bloc est répliqué sur un petit nombre (**facteur de réplication**) de machines physiquement séparées. (*par défaut trois*)



CC-BY NC SA

47

Hadoop: Architecture de HDFS

❑ Maître: NameNode

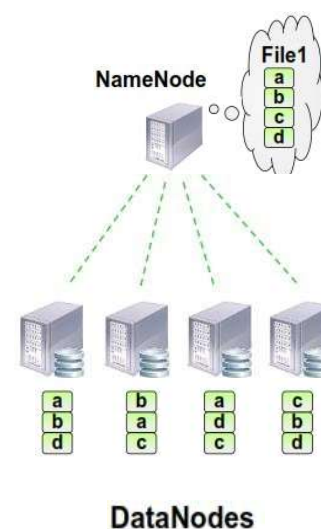
- Gère l'**espace de noms du système de fichiers**: Il gère l'arborescence du système de fichiers et les métadonnées de tous les fichiers (*noms, facteurs de réplication, propriétés*) et répertoires de l'arborescence.

Ces informations (**metadata**) sont stockées sur le disque local dans deux fichiers:

o**EditLog** : pour enregistrer chaque modification apportée aux métadonnées du système de fichiers (les nouveaux changements !). Par exemple, création d'un nouveau fichier dans HDFS, le NameNode insère un enregistrement dans le **EditLog** indiquant cela. De même, la modification du facteur de réplication d'un fichier entraîne l'insertion d'un nouvel enregistrement dans **EditLog**

o**FsImage** : (*métadonnées*) chargé en **mémoire** au démarrage du **NameNode**, (*la trace des changements depuis le début!*)

- Régule l'accès aux fichiers par les clients (équilibrer la charge)

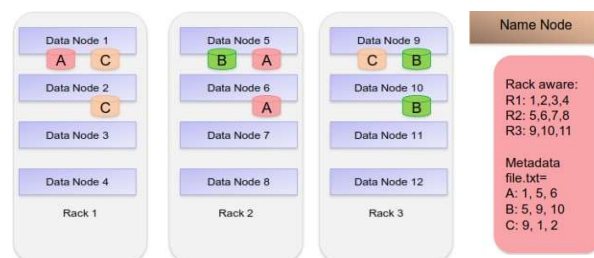


48

Hadoop: Architecture de HDFS

- Le **NameNode** conserve les **métadonnées** en **mémoire** pour assurer un accès rapide, ce qui nécessite une grande taille de mémoire pour son fonctionnement.
- Les blocs de fichiers sont répliqués sur plusieurs nœuds (**DataNodes**).
- Le nombre de copies de chaque bloc est contrôlé par le paramètre de réplication (par défaut 3)
- Réplication sur plusieurs racks:

Une copie sur un premier rack - La 2ème et la 3ème copies peuvent être ensemble sur un autre Rack.

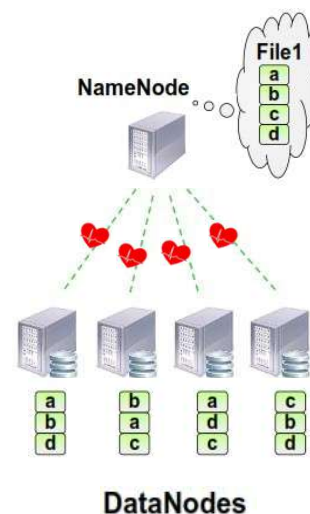


49

Hadoop: Architecture de HDFS

Esclaves: DataNode

- Nombreux DataNodes par cluster.(un par nœud)
- Gère le stockage (read/write) attaché aux nœuds
- Envoie périodiquement un signal au **NameNode** sous forme de **heartbeat** (*id, espace stockage, espace utilisé, ...*) avec un intervalle indiqué dans **hdfs-site.xml** (**dfs.heartbeat.interval**, par défaut 3s).
- Renvoie périodiquement (**dfs.blockreport.intervalMsec** par défaut 21600000 Ms=6h (ou 3600000Ms=1h)) au NameNode un **rapport** sur les blocs qu'il stocke (id, taille, date de création, ...)
- Si un **DataNode** tombe en panne, ses blocs seront répliqués sur d'autres **DataNodes** (**timeout** = $2 * \text{heartbeat.recheck.interval}$ (5min par défaut) + $10 * \text{heartbeat.interval}$) = 630s par défaut
- Effectue également la **création**, la **suppression** et la **réplication** de blocs sur ordre du **NameNode**.
- Les données sont stockées sur plusieurs nœuds

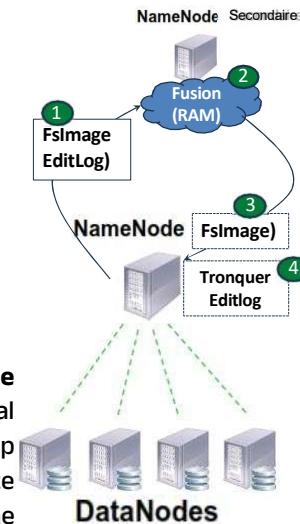


50

Hadoop: Architecture de HDFS

Secondary NameNode

- Le **NameNode** est un point de défaillance unique (*Single Point of Failure - SPOF*): Si la machine exécutant le **NameNode** est perdue, toutes les informations (**meta-data** dans la **RAM**) sur les fichiers de données seraient perdues. Par conséquent, on ne pourra pas localiser les blocs de fichiers sur les **DataNodes**.
- On récupère le **FsImage** en utilisant une **sauvegarde** et le fichier **EditLog**
- A cause de la **grande taille** du **EditLog**, le temps de récupération est **long**.
- On utilise un **NameNode secondaire** qui **fusionne périodiquement** l'image de l'espace des noms (**FsImage**) et le journal d'édition (**EditLog**) pour éviter que ce dernier devienne trop volumineux. Il s'exécute sur une machine physique distincte (*nécessite beaucoup de CPU et de RAM pour effectuer la fusion*). Il ne remplace pas le **NameNode**. (la mise à jour se fait presque chaque heure)



51

Hadoop: Architecture de HDFS

Secondary NameNode (suite)

- La **réplication** des métadonnées (FsImage) du **NameNode** et l'utilisation du **NameNode secondaire** permettent d'éviter la perte de métadonnées et de conserver une copie récente.
- Mais elle n'offre pas une **haute disponibilité** du système de fichiers:
 - En cas d'échec du **NameNode**, tous les clients, y compris les jobs MapReduce, ne pourraient pas lire, écrire ni répertorier les fichiers, car le **NameNode** est le seul référentiel des métadonnées et du mapping fichier-blocs.
 - Dans un tel cas, l'ensemble du système Hadoop serait **hors service** jusqu'à ce qu'un nouveau **NameNode** puisse être mis en ligne.
- Pour récupérer un **NameNode défaillant**, l'administrateur démarre un **nouveau NameNode** avec une copie récente du FsImage et configure les **DataNodes** et les applications clientes (*du Data Center*) pour utiliser ce nouveau **NameNode**.

52

Hadoop: Architecture de HDFS

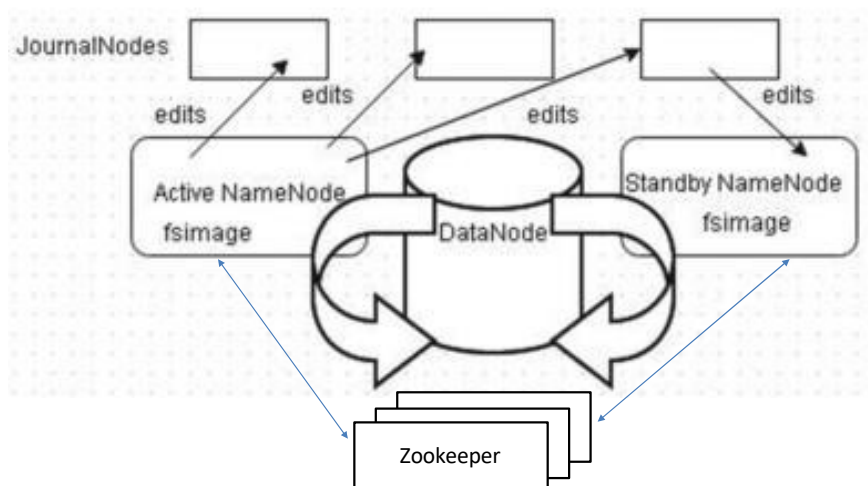
Secondary NameNode (suite)

- Le nouveau **NameNode** ne peut répondre aux demandes des clients tant qu'il n'a pas **(1)** chargé son **Fsimage dans la mémoire**, **(2)** relu son journal d'édition (EditLog), et **(3)** reçu suffisamment de rapports de blocs des **DataNodes** (*par défaut 95% des DN*) pour quitter le **mode sans échec (Safe Mode)**. [Paramètre: `dfs.namenode.safemode.threshold-pct`]
- Sur les grands clusters avec un grand nombre de fichiers et de blocs, le temps nécessaire pour qu'un **NameNode** démarre à froid peut prendre plusieurs minutes à plusieurs heures.
- **Hadoop 2** a remédié à cette situation en ajoutant la prise en charge de la **haute disponibilité (High Availability - HA)** HDFS: il y a un pair de **NameNodes Passif/Actif**. En cas de panne du **Namenode actif**, le **passif** devient **actif (à chaud)** pour continuer à répondre aux demandes des clients sans interruption significative.

53

Hadoop: Architecture de HDFS

Haute Disponibilité dans HDFS (high-availability – HA)



54

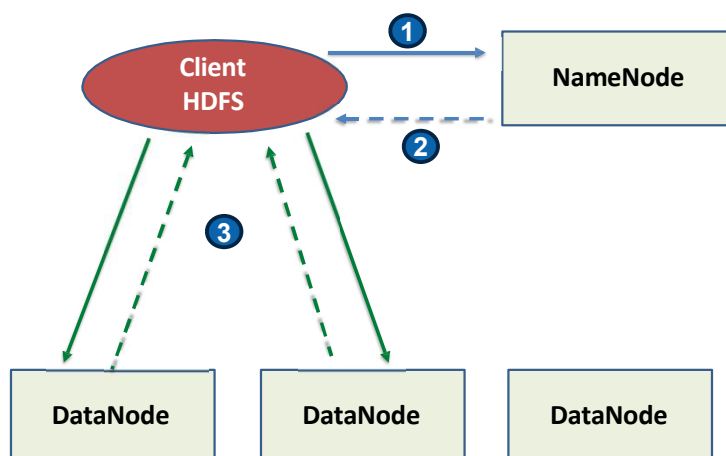
Hadoop: Architecture de HDFS

Haute Disponibilité dans HDFS (high-availability – HA) (Hadoop 2)

1. Le NameNode actif et le NameNode passif communiquent avec un groupe (quorum) de JournalNodes (généralement 3)
2. Le NameNode actif envoie les "edits" (logs) aux JournalNodes pour les partager avec le NameNode passif.
3. Le NameNode passif récupère les "edits" depuis l'un des JournalNodes et les applique à la copie Fsimage qu'il a, afin de la synchroniser avec celle du NameNode actif.
4. Les DataNodes envoient périodiquement les rapports sur les blocs aux NameNodes actif et passif.
5. ZooKeeper est un démon de coordination utilisé (*sur plusieurs nœuds*) pour coordonner le basculement rapide du NameNode actif vers le NameNode passif. ZooKeeper a deux fonctions dans un cluster HDFS haute disponibilité:
 - Détecter une défaillance du NameNode actif. Les deux NameNodes (passif et actif) conservent une session active dans ZooKeeper. Lorsqu'il y a défaillance du NameNode actif, ZooKeeper détecte cette défaillance suite à l'interruption de la session du NameNode actif.
 - ZooKeeper initie un basculement vers un NameNode passif et fournit un mécanisme pour l'**élection** du NameNode actif lorsqu'il y a plusieurs candidats (NameNodes passifs)

55

Hadoop: Lecture d'un fichier HDFS (1/4)



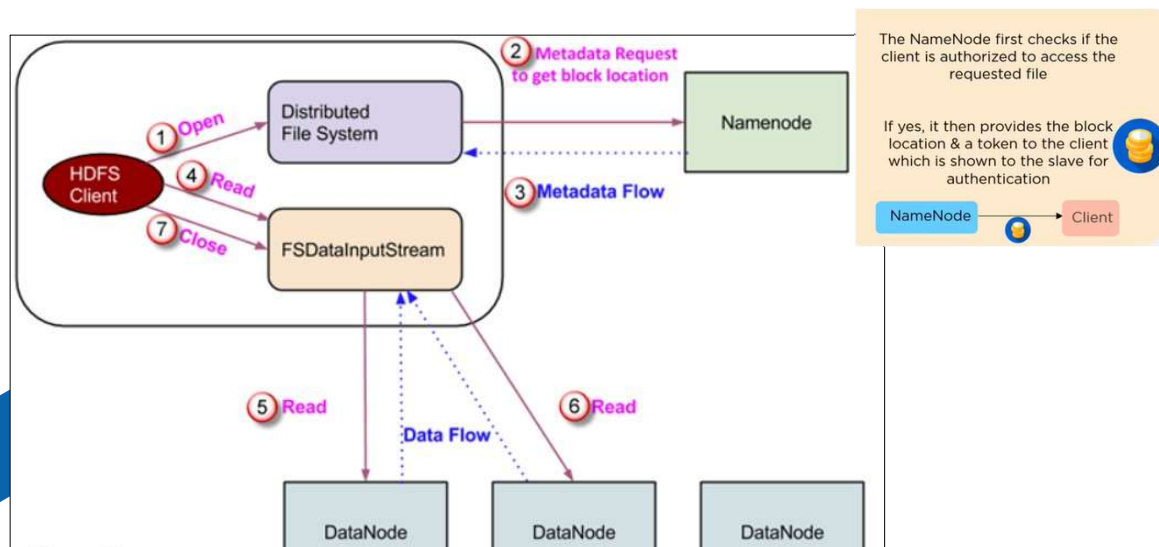
56

Hadoop: Lecture d'un fichier HDFS (2/4)

1. Le client (application HDFS) communique avec le NameNode pour demander la lecture d'un fichier (*ou plusieurs, un répertoire par exemple*)
2. Le NameNode envoie des métadonnées au client avec le détail du nombre de blocs, du nombre de répliquions, des identifiants et tailles des blocs ainsi que leur ordre et leurs emplacements.
3. Le client communique avec les DataNodes où les blocs sont présents.
4. Le client commence à lire les blocs en parallèle à partir des DataNodes en se basant sur les métadonnées reçues du NameNode.
5. Pour améliorer les performances de lecture, l'emplacement de chaque bloc est choisi en fonction de sa distance par rapport au client: d'abord lecture des blocs dans le même DataNode, puis un autre DataNode dans le même rack, et enfin un autre DataNode dans un autre rack.
6. Une fois que le client reçoit tous les blocs, il les combine pour former un fichier.

57

Hadoop: Lecture d'un fichier HDFS (3/4)



Référence: <https://www.guru99.com/learn-hdfs-a-beginners-guide.html>

58

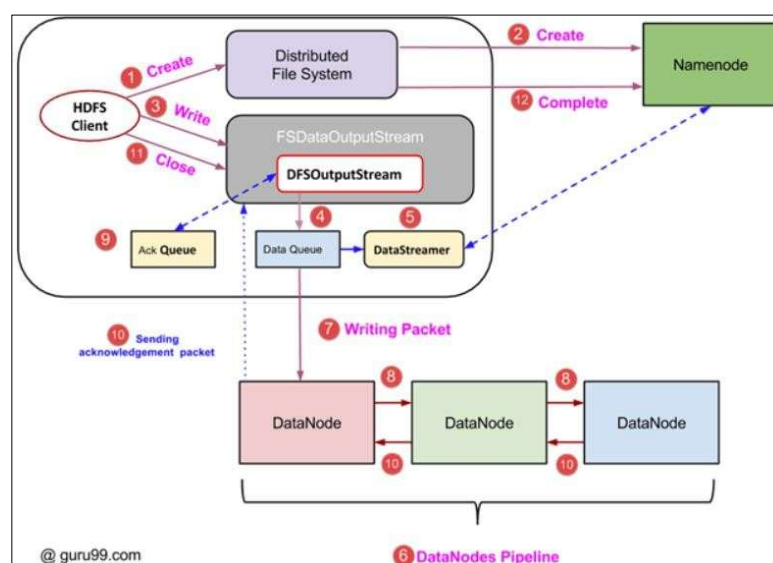
Hadoop: Lecture d'un fichier HDFS (4/4)

- (1) Le client appelle la méthode **open()** de l'objet **DistributedFileSystem** pour lire un fichier HDFS.
- (2) Cet objet se connecte au **NameNode** à l'aide d'un appel **RPC** pour avoir les métadonnées du fichier à lire telles que les emplacements des blocs du fichier.
- (3) Le **NameNode** envoie les métadonnées au client avec le détail du nombre de blocs, des identifiants et les tailles de blocs, des emplacements des blocs et du nombre de réplication.
- (4) Le client appelle la méthode **read()**, ce qui permet à un objet **FSDatalInputStream** d'établir une connexion avec un **DataNode** contenant un bloc du fichier
- (5)(6) Le client lit ce bloc sous forme de flux via des appels répétitifs de **read()**. A la fin d'un bloc, **FSDatalInputStream** ferme la connexion avec le **DataNode**.
- (7) La méthode **close()** est appelée à la fin de la lecture de tous les blocs.

Une fois que le client reçoit tous les blocs, il les combine pour former un fichier.

59

Hadoop: Ecriture d'un fichier dans HDFS (1/4)



Référence: <https://www.guru99.com/learn-hdfs-a-beginners-guide.html>

60

Hadoop: Ecriture d'un fichier dans HDFS (2/4)

1. Le client appelle la méthode **create()** de l'objet **DistributedFileSystem** qui permet de créer un nouveau fichier pour l'écriture.
2. Cet objet se connecte au **NameNode** à l'aide de **RPC** et lui demande la création d'un nouveau fichier. À ce stade, on n'associe aucun bloc à ce fichier. Le **NameNode** vérifie s'il y a déjà un autre fichier de même nom et si le client a les droits nécessaires pour créer le fichier. Dans ce cas, une entrée (dans le **FsImage** en mémoire) est créée pour le nouveau fichier. Dans le cas contraire, une exception **IOException** est soulevée.
3. Un objet de la classe **FSDDataOutputStream** est retourné au client afin de l'utiliser pour écrire le contenu du fichier dans **HDFS** en invoquant la méthode **write()** (en boucle)
4. L'objet **FSDDataOutputStream** utilise un autre objet **DFSOutputStream** qui gère la communication avec les **DataNodes** et le **NameNode**. **DFSOutputStream** crée à fur et à mesure des paquets (64k) avec les données écrites par le client. Ces paquets sont mis dans une file d'attente appelée **DataQueue**. **DFSOutputStream** décompose ainsi les fichiers à écrire en paquets.

61

Hadoop: Ecriture d'un fichier dans HDFS (3/4)

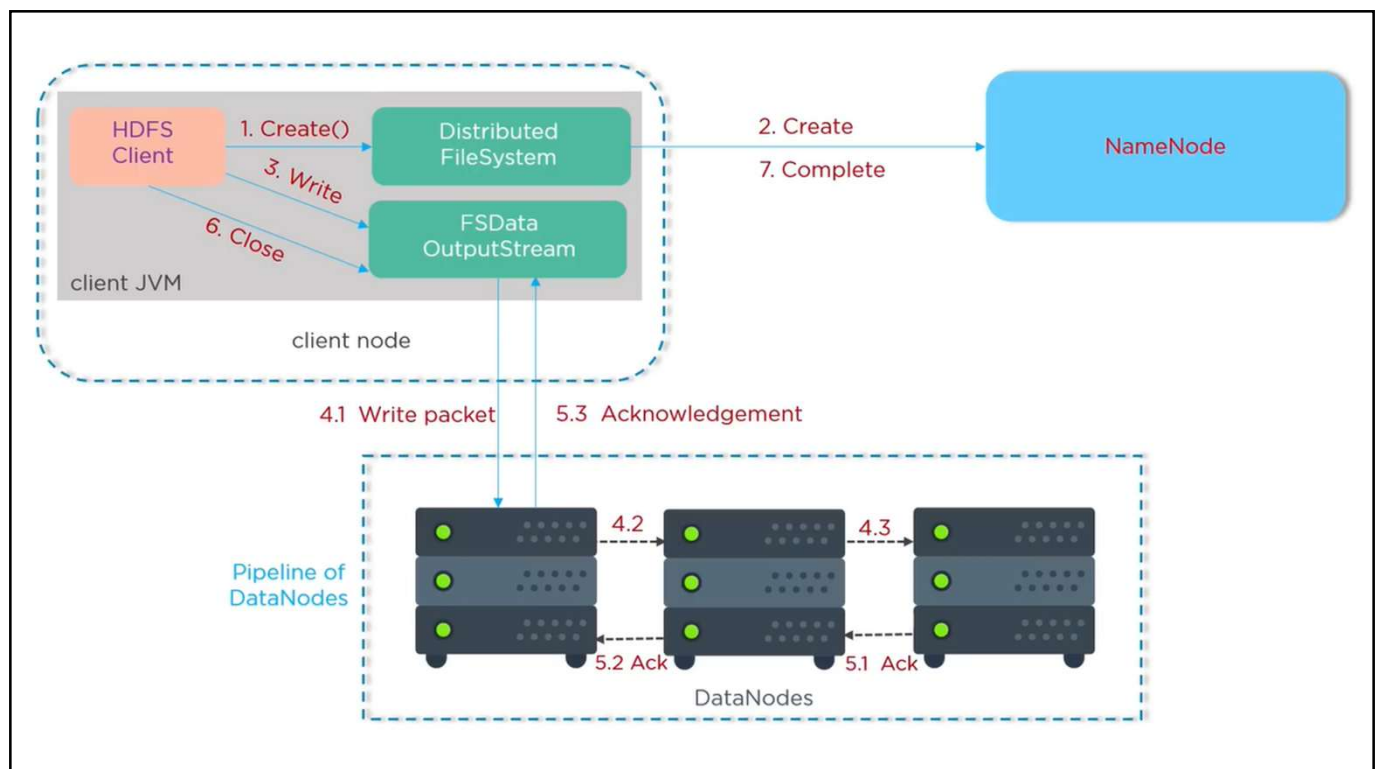
5. Le composant **DataStreamer** consomme les paquets de **DataQueue** et demande au **NameNode** l'allocation de nouveaux blocs. Le **NameNode** lui envoie les identifiants à utiliser pour les blocs, leurs emplacements et du nombre de réplication, ainsi les **DataNodes** à utiliser et leur ordre pour la réplication de chaque bloc seront identifiés.
6. Le processus de réplication d'un **bloc** commence par la création d'un pipeline (*par le client*) à l'aide de **DataNodes** identifiés dans l'étape 5. Dans le schéma, le facteur de réplication est 3, il y a donc 3 **DataNodes** dans le pipeline.
7. Le **DataStreamer** envoie des paquets vers le premier **DataNode** du pipeline
8. Chaque **DataNode** d'un pipeline stocke le paquet reçu et le transmet au **DataNode** suivant du pipeline.
9. **DFSOutputStream** gère une autre file d'attente, "**Ack Queue**", pour stocker les paquets qui attendent un accusé de réception de **DataNodes**.

62

Hadoop: Ecriture d'un fichier dans HDFS (4/4)

10. Lorsque l'accusé de réception d'un paquet dans la file d'attente est reçu de tous les DataNodes du pipeline, il est supprimé de la file «Ack Queue». En cas d'échec de DataNode, les paquets de cette file d'attente sont utilisés pour relancer l'opération.
11. Le client termine l'écriture des données par l'appel de la méthode `close()`, ce qui entraîne l'envoi des paquets de données restants vers le pipeline, puis l'attente de l'accusé de réception.
12. Une fois l'accusé de réception final reçu, le NameNode est contacté pour lui indiquer que l'opération d'écriture de fichier est terminée.

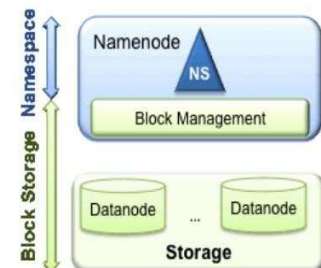
63



64

Architecture de la fédération HDFS (1/2)

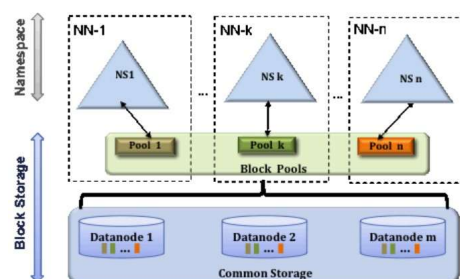
- Les fichiers dans **Hadoop (HDFS)** sont divisés en blocs qui sont ensuite stockés dans différents **DataNodes**.
- Pour accéder au contenu d'un fichier, on doit localiser ses blocs pour pouvoir les lire. Le **NameNode** permet cette localisation par la gestion de toutes les **méta-données** des fichiers et leurs blocs.
- Ces **méta-données** sont organisées en **pools de blocs (Blockpools)** et stockées dans la RAM du **NameNode**. Ainsi, on a besoin de les reconstituer en cas de perte (*Edit-Log, NameNode Secondaire, Rapports détaillés des DataNodes, Architecture HA*).
- Dans **Hadoop 1** on a un seul **NameNode** qui répond aux commandes d'accès aux fichiers de tous les utilisateurs.
- **Hadoop 2** a introduit la **fédération HDFS** basée sur l'utilisation de **plusieurs NameNodes**.



65

Architecture de la fédération HDFS (2/2)

- Dans une architecture de la **Fédération HDFS (Hadoop Federation)**, on a **multiple namespaces** par décomposition de l'arborescence des fichiers en plusieurs parties.
- **Exemple**: utilisation de trois namespaces: **/finance**, **/agriculture** et **/automobile**
- Le **Blockpool (métadonnées)** de chaque **namespace** est géré par un **NameNode** spécifique.
- Pour accéder à un fichier spécifique, on s'adresse à un **NameNode** bien déterminé.
- Chaque DataNode s'enregistre auprès de tous les NameNodes (*leur envoie périodiquement des pulsations et des rapports de blocs, exécute leurs commandes*)
- Les **Blockpools** sont gérés indépendamment les uns des autres. Ainsi, la perte d'un **NameNode** ne cause pas l'arrêt total de l'accès au **cluster Hadoop**.
- Avantages de la fédération HDFS sont:
 1. **Scalabilité et Performance**: l'ajout de NameNodes supplémentaires au cluster augmente le débit des opérations de lecture et écriture du système de fichiers.
 2. **Isolation**: On n'a aucune isolation avec un seul **NameNode** dans un environnement multi-utilisateur. Les tests d'une application peuvent surcharger le NameNode et ralentir les applications en production. L'utilisation de plusieurs NameNodes, des catégories d'applications et d'utilisateurs peuvent être isolées dans différents namespaces.



66

Hadoop: Modes de fonctionnement

Hadoop possède trois modes d'exécution :

- **Mode local (standalone)** : Hadoop fonctionne sur une seule machine et tout s'exécute dans la même JVM (*Java Virtual Machine*). En mode local le système de gestion de fichiers utilisé est celui du système hôte (Exemple Linux: ext3, ext4 ou xfs) et non HDFS.
- **Mode pseudo-distribué**: Hadoop fonctionne sur une seule machine, mais chacun des daemons principaux s'exécute dans sa propre JVM. Le système de fichier utilisé est HDFS dans ce mode.
- **Mode totalement distribué**: c'est le mode d'exécution réel d'Hadoop. Il permet de faire fonctionner le système de fichiers distribué et les daemons sur un ensemble de machines différentes.

67

Hadoop: Commandes HDFS

- Deux façons pour manipuler HDFS
 - via l'API Java
 - via les commandes depuis un terminal
- Nous utiliserons Hadoop avec un seul nœud pour tester et utiliser quelques commandes HDFS.
- Les clients HDFS utilisent la propriété **fs.default.name** ou **fs.defaultFS** de Hadoop pour déterminer l'URL de l'hôte et le port du **NameNode**.
- Les propriétés et paramètres Hadoop sont dans des fichiers de configuration:
 - /etc/hadoop/conf/**core-site.xml**
 - /etc/hadoop/conf/**hdfs-site.xml**:

Facteur de réplication (**dfs.replication**),
NameNode secondaire (**dfs.namenode.secondary.http-address**), ...

- Nous l'utiliserons sur **localhost** comme **fs.defaultFS = hdfs://localhost: 8020** avec le port **8020** HDFS par défaut, **fs.defaultFS = s3a://bucket_name** pour le système de fichier *Amazon Simple Storage Service* (S3), ou autre.

.....

68

Hadoop: Ligne de commande HDFS (1/3)

- Syntaxe utilisée: `$ hadoop fs -commande`

ou

`$ hdfs dfs -commande`

fs : Interpréteur (Shell) de commandes qui supporte plusieurs commandes: `mkdir`, `ls`, `copyFromLocal`,

- Pour avoir la liste des commandes: `$ hdfs dfs -help`
- Pour avoir la description d'une commande: `$ hdfs dfs -help commande`
- Pour avoir la syntaxe d'utilisation d'une commande: `$ hdfs dfs -usage commande`

Exemple: `$ hdfs dfs -help mkdir`

Exemple: `$ hdfs dfs -usage put`

- Pour vérifier l'état du contenu d'un dossier HDFS: `$ hdfs fsck chemin -options`

Exemples: `$ hdfs fsck /user/tp -files`

`$ hdfs fsck /user/tp -files -blocks -locations`

`$ hdfs fsck /user/tp/file1.csv -blocks -locations`

- Pour avoir la syntaxe détaillée de `hdfs fsck`: `$ hdfs fsck -help`

69

Hadoop: Ligne de commande HDFS (2/3)

Exemples:

- Afficher l'arborescence de la racine HDFS: `$ hdfs dfs -ls /`

Ou `$ hdfs dfs -ls hdfs://localhost:8020/`

- Créer un dossier dans HDFS: `$ hdfs dfs -mkdir /user/amine`

- Copier un fichier local vers HDFS:

`-put`

`$ hdfs dfs -copyFromLocal /home/cloudera/file1.txt /user/amine/file1.txt`

- Copier le contenu d'un dossier local vers HDFS:

`$ hdfs dfs -put /home/cloudera/bank /user/amine/`

- Copier un fichier vers HDFS en précisant la taille de bloc différente de celle par défaut:

`$ hdfs dfs -D dfs.blocksize=33554432 -put /home/cloudera/file1.txt /user/amine/file1.txt`

70

Hadoop: Ligne de commande HDFS (3/3)

Exemples:

- Afficher le contenu d'un dossier HDFS:

```
$ hdfs dfs -ls /user/amine
```

- Copier un fichier HDFS vers le système local

```
$ hdfs dfs -get  
-copyToLocal /user/amine/quangle.txt /home/cloudera/quangle.txt
```

- Afficher les dernières lignes (1Ko) d'un fichier:

```
$ hdfs dfs -tail /user/amine/quangle.txt
```

- Afficher un rapport sur les blocs des fichiers d'un dossier donné:

```
$ hdfs fsck /user/amine-files -blocks -locations
```

- Avoir l'emplacement du FsImage: **hdfs getconf -confKey dfs.namenode.name.dir**